```
"""Loads free historical results from football-data.co.uk for the chosen leagues.
import io
import urllib.request

import pandas as pd

BASE = "https://www.football-data.co.uk"
SEASONS = ["2324", "2425", "2526", "2627"]  # oldest to newest; missing ones are

# name -> (kind, code). "main" files are per season; "extra" files hold all seaso
LEAGUES = {
    "England - Premier League": ("main", "E0"),
    "England - Championship": ("main", "E1"),
    "England - League One": ("main", "E2"),
    "England - League Two": ("main", "E3"),
    "England - National League": ("main", "EC"),
    "Spain - La Liga": ("main", "SP1"),
    "Spain - Segunda": ("main", "SP2"),
    "Germany - Bundesliga": ("main", "D1"),
    "Germany - 2. Bundesliga": ("main", "D2"),
    "France - Ligue 1": ("main", "F1"),
    "France - Ligue 2": ("main", "F2"),
    "Netherlands - Eredivisie": ("main", "N1"),
    "Turkey - Super Lig": ("main", "T1"),
    "Norway - Eliteserien": ("extra", "NOR"),
    "Mexico - Liga MX": ("extra", "MEX"),
}


def _download(url: str) -> pd.DataFrame | None:
    try:
        req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
        with urllib.request.urlopen(req, timeout=20) as resp:
            raw = resp.read()
        return pd.read_csv(io.BytesIO(raw), encoding="latin-1", on_bad_lines="ski
    except Exception:
        return None


def standardise(df: pd.DataFrame) -> pd.DataFrame:
    """Rename columns to Date, Home, Away, HG, AG and drop unplayed or broken row
    df = df.rename(columns={"HomeTeam": "Home", "AwayTeam": "Away", "FTHG": "HG",
    needed = ["Date", "Home", "Away", "HG", "AG"]
    if not all(c in df.columns for c in needed):
```

```
        return pd.DataFrame(columns=needed)
    df = df[needed].copy()
    df["Date"] = pd.to_datetime(df["Date"], dayfirst=True, errors="coerce")
    df["HG"] = pd.to_numeric(df["HG"], errors="coerce")
    df["AG"] = pd.to_numeric(df["AG"], errors="coerce")
    return df.dropna().reset_index(drop=True)


def load_league(name: str, years_back: float = 3.0) -> pd.DataFrame:
    kind, code = LEAGUES[name]
    if kind == "main":
        frames = [_download(f"{BASE}/mmz4281/{s}/{code}.csv") for s in SEASONS]
    else:
        frames = [_download(f"{BASE}/new/{code}.csv")]
    frames = [standardise(f) for f in frames if f is not None]
    if not frames:
        return pd.DataFrame(columns=["Date", "Home", "Away", "HG", "AG"])
    df = pd.concat(frames, ignore_index=True).drop_duplicates(["Date", "Home", "A
    cutoff = df["Date"].max() - pd.Timedelta(days=int(365 * years_back))
    return df[df["Date"] >= cutoff].sort_values("Date").reset_index(drop=True)
```